19

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

인젝션 스코프

제2부 NestJS 핵심 · 5장 의존성 주입

서버에 하나뿐인 서비스와 요청마다 다른 값



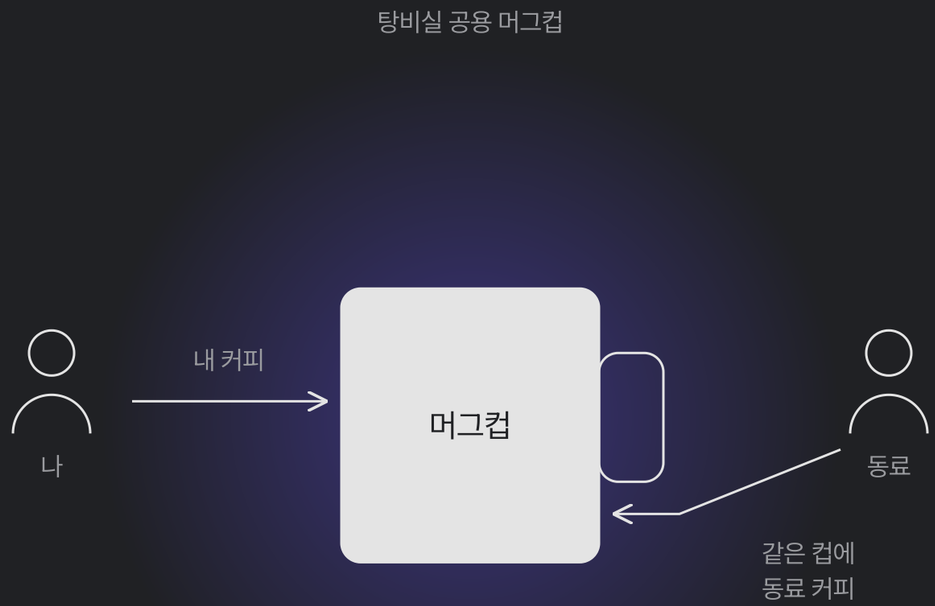
© 한상훈 NestJS 백엔드 마스터

사무실 공용 머그컵

→ 탕비실 머그컵 하나, 직원 여럿

→ 내가 탄 커피 → 자리 비운 사이 다른 사람이 같은 컵에 커피

→ 돌아와서 마신 커피: 남의 커피



돌아와서 마신 커피: 남의 커피

© 한상훈 NestJS 백엔드 마스터

같이 쓰려고 둔 필드

→ 서비스: 기본은 서버에 하나, 모든 요청이 같은 객체

→ 좌석표 Map: 모두가 같이 보려고 둔 필드

→ 요청 하나만의 값(구매자, 로그인 사용자)을 필드에 두면?

© 한상훈 NestJS 백엔드 마스터

필드에 넣은 구매자

reservations.service.ts

```ts
@Injectable()
export class ReservationsService {
  private buyer: string;

  constructor(private readonly gateway: PaymentGateway) {
    console.log('ReservationsService 생성');
  }

  async reserve(seatId: string, amount: number, buyer: string) {
    this.buyer = buyer;
    const payment = await this.gateway.charge(amount);
    return { seatId, ...payment, notifyTo: this.buyer };
  }
}
```

→ notifyTo: 예매 확정 문자를 받을 사람

© 한상훈 NestJS 백엔드 마스터

거의 동시에 들어온 두 예매

```
// race.mjs
const a = send({ seatId: 'A-3-15', amount: 110000, buyer: '김민수' });
await new Promise((r) => setTimeout(r, 50));
const b = send({ seatId: 'B-7-2', amount: 99000, buyer: '이지은' });
console.log(await a);
console.log(await b);
```

→ 가짜 결제사 응답 0.2초 지연(FAKE_PG_DELAY_MS=200)

→ 0.05초 간격의 두 요청

© 한상훈 NestJS 백엔드 마스터

김민수의 예매, 이지은에게 간 문자

터미널

```
$ node race.mjs 4019
{"seatId":"A-3-15","approvalId":"fake-110000","amount":110000,"notifyTo":"이지은"}
{"seatId":"B-7-2","approvalId":"fake-99000","amount":99000,"notifyTo":"이지은"}
```

→ 좌석과 금액은 맞음, 받는 사람만 틀림

→ 에러 없음, 테스트 통과(요청 하나씩이면 정상)

© 한상훈 NestJS 백엔드 마스터

기다리는 사이의 틈

→ 0.00초: 김민수 요청, `this.buyer = 김민수`, `await charge`

→ 0.05초: 이지은 요청, `this.buyer = 이지은`, `await charge`

→ 0.20초: 김민수 결제 끝, `this.buyer` 읽기 → 이지은

→ 0.25초: 이지은 결제 끝, `this.buyer` 읽기 → 이지은

"await 한 번마다 다른 요청이 끼어듭니다.
읽고, 기다리고, 쓰는 사이가 전부 틈입니다."

© 한상훈 NestJS 백엔드 마스터

스코프 세 가지

스코프	만드는 때	몇 개
DEFAULT	부팅 때	서버에 하나
REQUEST	요청이 올 때	요청마다 하나
TRANSIENT	주입받는 곳이 생길 때	주입받는 곳마다 하나

→ 머그컵으로: DEFAULT 탕비실 컵 하나 / REQUEST 커피마다 새 종이컵 / TRANSIENT 직원마다 이름 적힌 컵

© 한상훈 NestJS 백엔드 마스터

공식 문서의 정의

→ 인젝션 스코프(injection scope): 프로바이더 인스턴스를 언제, 몇 개 만들지

스코프	공식 문서 문장
DEFAULT	"A single instance of the provider is shared across the entire application."
REQUEST	"A new instance of the provider is created exclusively for each incoming request."
TRANSIENT	"Transient providers are not shared across consumers."

데이터 블록: docs.nestjs.com/fundamentals/injection-scopes, 2026-10-26 기준, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

Scope.REQUEST

reservations.service.ts

```ts
import { Injectable, Scope } from '@nestjs/common';

@Injectable({ scope: Scope.REQUEST })
export class ReservationsService {
  private buyer: string;
  ...
}
```

→ 필드 코드는 그대로, 스코프만 변경

© 한상훈 NestJS 백엔드 마스터

제대로 돌아온 구매자

터미널

```
$ node race.mjs 4019
{"seatId":"A-3-15","approvalId":"fake-110000","amount":110000,"notifyTo":"김민수"}
{"seatId":"B-7-2","approvalId":"fake-99000","amount":99000,"notifyTo":"이지은"}
```

© 한상훈 NestJS 백엔드 마스터

요청마다 생기는 객체

부팅 로그와 예매 요청 두 번

```
LOG [NestFactory] Starting Nest application...
ConcertsService 생성
FakePaymentGateway #1 생성
SeatsService 생성
RefundsService 생성
LOG [NestApplication] Nest application successfully started
ReservationsService 생성
ReservationsController 생성
ReservationsService 생성
ReservationsController 생성
```

→ 부팅 때: 예매 서비스, 예매 컨트롤러 없음

→ 예매 요청 두 번: 서비스와 컨트롤러가 두 번씩

© 한상훈 NestJS 백엔드 마스터

환불 요청에도 생긴 예매 서비스

터미널

```
$ curl -X POST localhost:4019/reservations/refund -d '{"approvalId":"fake-110000"}'
{"refunded":"fake-110000"}

ReservationsService 생성
ReservationsController 생성
```

→ 환불 메서드: 예매 서비스를 쓰지 않음

→ 컨트롤러가 요청마다 생성 → 생성자 인자인 예매 서비스도 생성

© 한상훈 NestJS 백엔드 마스터

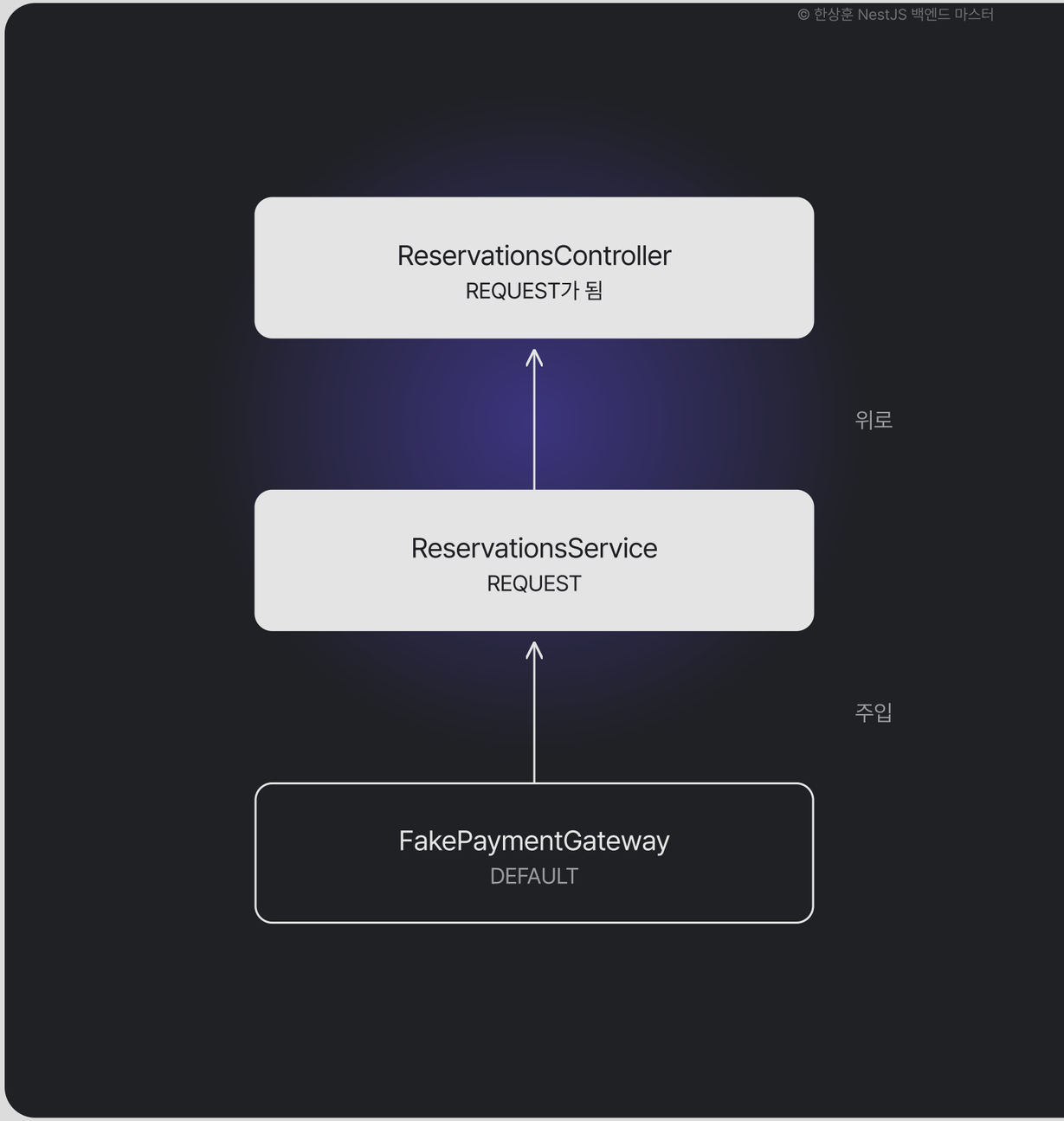
위로 번지는 스코프

→ `FakePaymentGateway` (DEFAULT)

→ ← `ReservationsService` (REQUEST)

→ ← `ReservationsController` (REQUEST가 됨)

→ 받는 쪽이 REQUEST를 받으면 받는 쪽도 REQUEST



데이터 블록: "The REQUEST scope bubbles up the injection chain." (docs.nestjs.com/fundamentals/injection-scopes, 2026-10-26 기준), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

번지는 범위

→ 로거 하나를 REQUEST로: 그 로거를 받는 모든 서비스와 컨트롤러가 REQUEST

→ 요청마다 만드는 객체 수 = 사슬 위쪽 전부

→ 부팅 때 만들어 두는 객체 감소, 부팅 때 발견하던 문제의 일부가 요청 때로

© 한상훈 NestJS 백엔드 마스터

처리량 측정

스코프	초당 처리 (5회)
DEFAULT	5,657~7,245
REQUEST	4,823~7,681

→ 동시 100개, 요청 3만 개, 로그 출력 끔, 같은 PC에서 측정 도구와 서버 동시 실행

→ 차이가 측정 흔들림보다 작음: 이 크기의 사슬에서는 숫자로 안 보임

데이터 블록: DEFAULT 7,245 / 6,660 / 5,657 / 6,213 / 6,534, REQUEST 6,892 / 6,319 / 7,681 / 4,823 / 7,022, 2026-10-26 강사 PC 실측(Node v24.11.0), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

공식 문서의 권고

→ "Request-scoped providers affect application performance."

→ "Unless a provider must be request-scoped, we strongly recommend using the default singleton scope."

→ Passport 전략, Cron 컨트롤러: 싱글톤이어야 하는 프로바이더

데이터 블록: docs.nestjs.com/fundamentals/injection-scopes, 2026-10-26 기준, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

구매자는 인자로

reservations.service.ts

```ts
@Injectable()
export class ReservationsService {
  constructor(
    private readonly gateway: PaymentGateway,
    private readonly logger: TaggedLogger,
  ) {
    this.logger.setTag('예매');
  }

  async reserve(seatId: string, amount: number, buyer: string) {
    const payment = await this.gateway.charge(amount);
    this.logger.log(`${seatId} 결제 완료`);
    return { seatId, ...payment, notifyTo: buyer };
  }
}
```

```
{"seatId":"A-3-15","approvalId":"fake-110000","amount":110000,"notifyTo":"김민수"}
{"seatId":"B-7-2","approvalId":"fake-99000","amount":99000,"notifyTo":"이지은"}
```

→ 필드 삭제, 다시 DEFAULT

→ 요청 하나의 값: 인자와 지역 변수

© 한상훈 NestJS 백엔드 마스터

REQUEST 스코프가 맞는 경우

→ 요청 정보가 사슬 깊은 곳까지 필요할 때

→ 예: 요청마다 다른 테넌트 DB 연결, 요청 객체(REQUEST 토큰) 주입

→ 인자로 다섯 단계를 넘기기 어려울 때의 선택지

→ 그때도 번지는 범위 확인

© 한상훈 NestJS 백엔드 마스터

공용 로거에 붙인 태그

tagged-logger.ts

```
@Injectable()
export class TaggedLogger {
  private tag = '';

  setTag(tag: string) {
    this.tag = tag;
  }

  log(message: string) {
    console.log(`[${this.tag}] ${message}`);
  }
}
```

→ 예매 서비스 생성자: `setTag('예매')`

→ 환불 서비스 생성자: `setTag('환불')`

© 한상훈 NestJS 백엔드 마스터

예매 로그에 찍힌 환불

서버 로그

```
[환불] A-3-15 결제 완료
[환불] B-7-2 결제 완료
[환불] fake-110000 환불
```

→ 로거는 서버에 하나, 마지막 setTag가 '환불'

→ 장애 때 ' [예매] ' 로 검색하면 0건

© 한상훈 NestJS 백엔드 마스터

Scope.TRANSIENT

tagged-logger.ts

```
@Injectable({ scope: Scope.TRANSIENT })
export class TaggedLogger {
  static created = 0;
  private tag = '';

  constructor() {
    console.log(`TaggedLogger #${++TaggedLogger.created} 생성`);
  }
```

© 한상훈 NestJS 백엔드 마스터

로거 두 개

서버 로그

```
TaggedLogger #1 생성
TaggedLogger #2 생성
FakePaymentGateway #1 생성
...
[예매] A-3-15 결제 완료
[예매] B-7-2 결제 완료
[환불] fake-110000 환불
```

→ 부팅 때 두 개: 예매 서비스용, 환불 서비스용

→ 요청 중 추가 생성 없음, 받는 쪽 스코프도 그대로

© 한상훈 NestJS 백엔드 마스터

세 스코프 비교

스코프	이 프로젝트의 예	필드에 둘 것
DEFAULT	결제사 객체, 예매 서비스	모든 요청이 같이 써도 되는 것
REQUEST	(이번 강 시연에서만)	그 요청의 값
TRANSIENT	TaggedLogger	주입받은 곳의 값

© 한상훈 NestJS 백엔드 마스터

싱글톤 필드

→ 결제사 객체, 좌석표: 같이 쓰려고 둔 필드

→ 구매자, 태그: 요청이나 쓰는 곳마다 다른 값

"싱글톤 필드에는 모든 요청이 같이 써도 되는 것만 둡니다."